

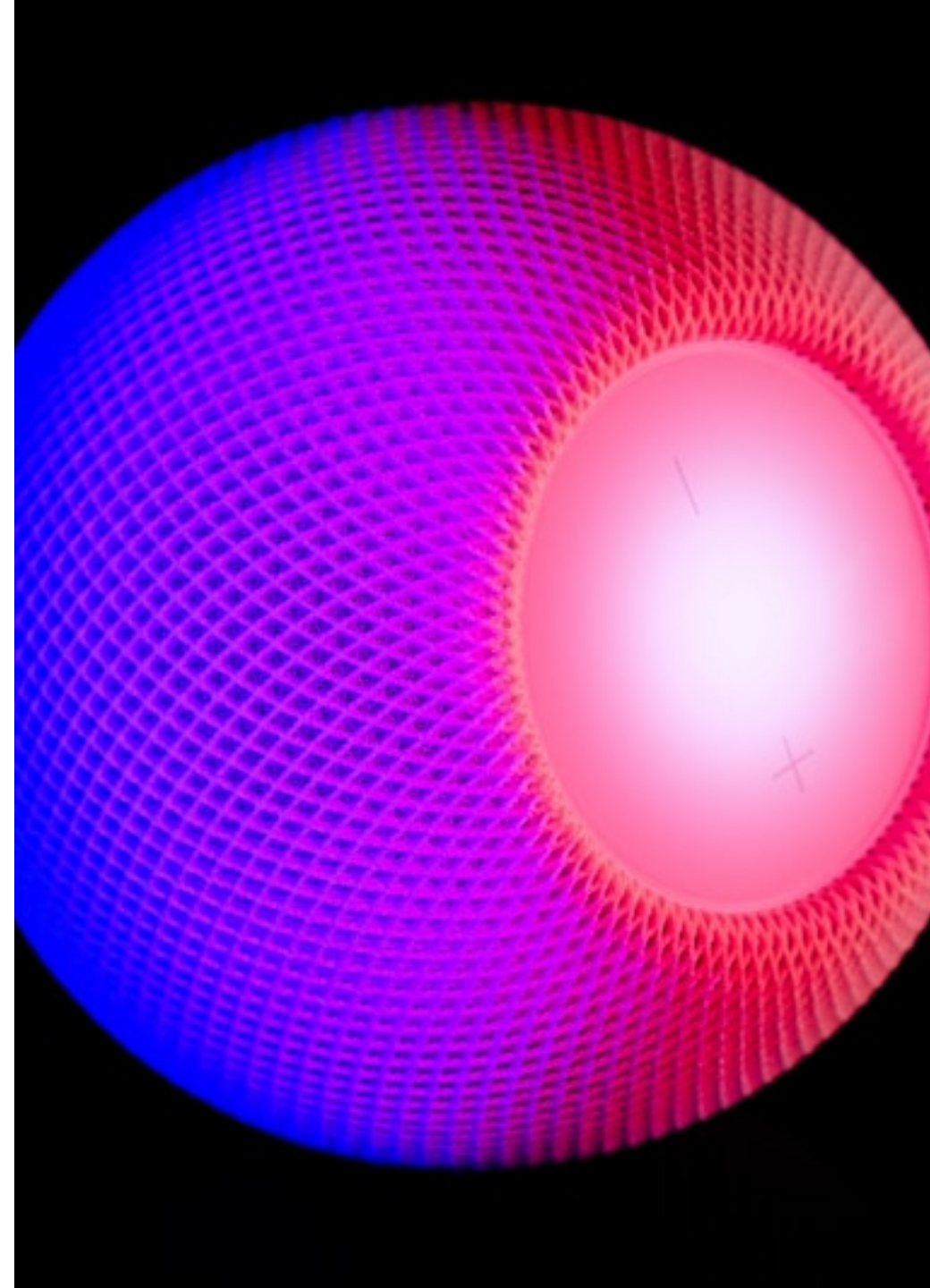


L18: Objects and Methods

Drone Calibration Protocol

Neon City Cyber Academy

Module 5: Object-Oriented Programming



Your Team

Emily Chen - *Handler*

"Welcome to the Drone Division. Today you're learning about **objects** - the foundation of modern programming."

Cole Martinez - *Tech Lead*

"Every turtle is an object. Every game character is an object. Time to understand why OOP runs the world."

What is an Object?

Objects are things in code that have:

1. **Data** (attributes/properties)
2. **Actions** (methods/functions)

Real World Example:

Drone Object:

- Data: Name, Battery Level, Position
- Actions: `fly()` , `land()` , `recharge()`

In Python, **EVERYTHING** is an object!

The Turtle is an Object!

```
import turtle

# Create a turtle OBJECT
drone = turtle.Turtle()
```

What just happened?

- `turtle.Turtle()` is a **class** (blueprint)
- `drone` is an **instance** (actual object)
- Like building a drone from factory specs!

Methods = Actions

Methods are functions that belong to objects:

```
drone = turtle.Turtle()

# Call methods on the object
drone.forward(100)    # Move forward
drone.right(90)       # Turn right
drone.color('cyan')   # Change color
```

Syntax: `object.method(parameters)`

The dot (`.`) means "use this object's ability"

Emily Explains:

"Think of an object like a remote-controlled drone:"

- The **drone** = object
- The **remote buttons** = methods
- Pressing `forward()` button = calling a method

"You don't need to know HOW the drone flies internally. You just press the button!"

This is called **ABSTRACTION** - hiding complexity.



Task 1: Basic Calibration

```
import turtle

# Activate navigation unit
drone = turtle.Turtle()

# Execute calibration sequence
drone.forward(100)
drone.right(90)
drone.forward(100)

turtle.done()
```

Creates an L-shaped flight path!

Customizing Your Drone

You can change how your drone looks:

```
neon_drone = turtle.Turtle()

# Appearance methods
neon_drone.shape('turtle') # Change shape
neon_drone.color('cyan')   # Change color
neon_drone.pensize(3)       # Change line width
neon_drone.speed(5)         # Change speed (1-10)
```

Each method changes a property of the object!

Cole's Pro Tips:

Common Turtle Shapes:

- `'turtle'` - Classic turtle
- `'circle'` - Drone-like
- `'square'` - Pixelated
- `'arrow'` - Directional

Speed Settings:

- `0` = Instant (no animation)
- `1` = Slowest
- `10` = Fastest

Chaining Commands

You can call multiple methods in sequence:

```
drone = turtle.Turtle()
drone.shape('circle')
drone.color('lime')
drone.speed(2)

# Draw a square patrol zone
for i in range(4):
    drone.forward(100)
    drone.right(90)
```

Each method modifies the **SAME** object!

Task 2: Neon Drone

```
import turtle

# Initialize and customize
neon_drone = turtle.Turtle()
neon_drone.shape('turtle')
neon_drone.color('cyan')

# Execute patrol pattern
for i in range(4):
    neon_drone.forward(100)
    neon_drone.right(90)

turtle.done()
```

Cyan square - Neon City style!

Advanced: Multiple Drones

You can create MULTIPLE objects from the same class:

```
# Two independent drones
drone1 = turtle.Turtle()
drone1.color('red')

drone2 = turtle.Turtle()
drone2.color('blue')

# Each moves independently!
drone1.forward(50)
drone2.backward(50)
```

Same blueprint, different instances!

Emily's Insight:

"This is the power of Object-Oriented Programming:"

- **One blueprint** (the Turtle class)
- **Many objects** (drone1, drone2, drone3...)
- Each with **its own data** (position, color)
- All sharing **the same methods** (forward, right)

"Like a factory producing many drones from one design document!"

Drawing Shapes with Objects

```
# Hexagon drone
advanced_drone = turtle.Turtle()
advanced_drone.shape('circle')
advanced_drone.color('lime')

# 6-sided patrol route
for side in range(6):
    advanced_drone.forward(80)
    advanced_drone.right(60)

turtle.done()
```

Hexagons: $360^\circ \div 6 \text{ sides} = 60^\circ \text{ turns}$



Task 3: Hexagon Patrol

Create an advanced navigation pattern!

Requirements:

- Shape: `'circle'` (drone-like)
- Color: `'lime'` (high-visibility)
- Pattern: Hexagon (6 sides, 80 units each)
- Turn angle: 60 degrees

Challenge: Can you make it draw a pentagon? (Hint: 5 sides, 72° turns)

Key Concepts Review

Objects Have:

- **State** (data/attributes): color, position, shape
- **Behavior** (methods): forward(), right(), color()

The Dot Operator:

```
object.method() # "Hey object, do this action!"
```

Creating Objects:

```
variable = ClassName() # Instance created
```


Cole's Object Rules:

Rule #1: An object is a container for data + functions

Rule #2: Methods need `()` parentheses to execute

Rule #3: The same class can create infinite objects

Remember: `drone.forward(50)` means:

"Call the `forward` method ON the `drone` object with parameter `50`"

Lesson Complete!

You Mastered:

-  What objects are (data + methods)
-  Creating turtle objects
-  Calling methods with dot syntax
-  Customizing object properties
-  Using multiple objects independently

Next Mission: L19 - Events

Make your code **INTERACTIVE** with keyboard and mouse!



Your Assignment

Create a Fleet Formation:

1. Create 3 turtle objects
2. Give each a different color
3. Make each draw a different shape
4. Use `goto(x, y)` to position them

Goal: Create a formation of drones in Neon City!